Safe to Drop: an Exact Criterion for Quantum Temporaries

ANONYMOUS AUTHOR(S)

We study when a quantum compiler can restore a temporary with one unitary on that register alone, without changing its computation on the other registers. We specify safety by observational equivalence to a program that prepares the temporary independently of the remaining computation, comparing their joint behavior immediately before the drop. In a typed language where every unitary is a valid gate, we characterize this specification by an exact test on the program's state map. Without measurement, the drop is safe exactly when the temporary ends in one fixed pure state for every input, and one local unitary then restores its allocation state. With measurement, the test applies to each record of outcomes and retains its probability weight. We also treat blocks of registers and give a return criterion for loans, registers borrowed from a caller. A restricted compiler implements restoration and wire reuse, which we test by simulation.

Additional Key Words and Phrases: quantum programming, contextual equivalence

1 Specifying a safe drop

We want to be able to reuse temporary registers once their lifetimes end, as we do classically by discarding their values. Unitary quantum gates are reversible, so we usually restore a temporary by uncomputation, running the operations that produced it backwards [12, §1]. The main nuance with reuse is that a temporary may go out of scope while still entangled with other registers. Their joint state then cannot be split into separate states. Uncomputation can undo these correlations, but may require operations on the other registers as well [12, §3, Fig. 2]. Measuring and resetting a temporary would also allow reuse, but that operation is nonunitary. We seek a restoration usable in code that must remain unitary, and that can also return an unknown borrowed register without erasing it. We ask when one unitary gate on the temporary alone can restore its allocation state and leave the rest intact. One gate means one unitary on all the registers dropped together, though it may need a hardware circuit.

Consider the measurement-free programs P, Q below. Program P allocates t in a pure state ψ, applies unitary gates M to t and input v, then drops t and returns v. Dropping removes a name and applies no physical operation. We compare its joint computation before the drop with an independent witness Q:

$$P(v) = \texttt{new t @ } \psi \texttt{; M; drop t; return v},$$
$$Q(v) = \texttt{new t @ } \psi \texttt{; R t; U v; drop t; return v}.$$

The drop is safe if fixed unitaries R and U satisfy $\forall x,\ M(\psi \otimes x) \simeq (R\psi) \otimes Ux$. Here $\simeq$ compares the joint states of t and v immediately before the drop of t. No subsequent gates and measurements can distinguish them. The same R and U must work for every input, with arbitrary unitaries available as gates. The instructions of M need not split into separate code for t and v; their action on inputs with t initialized in ψ must separate. In Q, the temporary ends in $R\psi$ whatever the input, so $R^\dagger$ restores it without acting on v. The equivalence lets a compiler insert the same restoration before P's drop. We first check this

specification on two programs without measurement. Each program below uses two-state quantum registers, or qubits, of type `[2]`. It allocates t in $|0\rangle$ and drops its name while keeping v. Both safe and unsafe cases are well typed. Instructions that differ from program (a) are shown in purple.

(a) *superpose, then drop*

```
prog a (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;

  drop t;
  return v }
```

(b) *superpose, then read*

```
prog b (v : [2]) : [2] {
  new t : [2] @ |0>;
  H t;
  CX t v;
  drop t;
  return v }
```

In program (a), the Hadamard gate H puts t in $|+\rangle = (|0\rangle + |1\rangle)/\sqrt{2}$. This is the same state whatever v held, and another H returns t to $|0\rangle$ without touching v. Program (b) adds `CX t v`, which flips v when t is 1. At input $|0\rangle_v$, this read entangles t and v. Their joint state is $(|00\rangle + |11\rangle)/\sqrt{2}$, which cannot be prepared by independent operations on the two registers. With t hidden, v is an equal mixture of $|0\rangle$ and $|1\rangle$. In both programs, t's lifetime ends at the drop, and neither uses t afterwards. Checking those lifetimes alone would accept both, though only program (a) permits this local unitary restoration. An unsafe drop is still legal in this language; it rules out this restoration on the temporary alone, rather than making the program ill typed.

2 Checking the specification

For programs without measurement, we can check the specification directly on the joint state map. We call one occurrence of `drop` a *site*, and the tuple of registers it names its *block*. Here each block is just t. A pure state of n qubits has 2^n complex coordinates, one per bit string. Let K map input x in v to the joint state just before the drop, with t first. The theorem below shows that the specification is equivalent to $Kx = g \otimes Wx$ for every input, where $g = R\psi$ is the temporary's fixed state and W is the computation on v, equal to U up to one overall phase. Observational equivalence determines each joint pure output up to phase, and linearity makes that phase the same at every input. Absorbing it into W lets us use equality here. For program (a), we can calculate this map directly:

$$x \xrightarrow{\texttt{new t @ |0>}} |0\rangle \otimes x \xrightarrow{\texttt{H t}} \frac{|0\rangle \otimes x + |1\rangle \otimes x}{\sqrt{2}} = |+\rangle \otimes x.$$

Similarly, program (b) has the same first two steps, followed by

$$|+\rangle \otimes x \xrightarrow{\texttt{CX t v}} \frac{|0\rangle \otimes x + |1\rangle \otimes Xx}{\sqrt{2}}.$$

Program (a) leaves v alone for both basis labels of the temporary. Program (b) leaves v alone for label 0 and applies the bit flip X for label 1.

Think of K as a `switch` on the basis labels of a temporary of dimension d. Branch i acts on the other registers through the *slice* K_i, formed by the rows labelled i (fig. 1). Pairing its result with the temporary's basis state $|i\rangle$ gives $Kx = \sum_i |i\rangle \otimes K_i x$. A classical switch

selects one branch. The quantum sum assembles all slices into Kx, the joint state before the drop, keeping their amplitudes and phases.

$$K_{(a)} = \overbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & -1 & 0 \\ 0 & 1 & 0 & -1 \end{bmatrix}}^{H \otimes I} \cdot \overbrace{\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}}^{|0\rangle \otimes I} = \overbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}}^{K_{(a)}} \qquad \overbrace{\frac{1}{\sqrt{2}}\begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 1 \\ 1 & 0 \end{bmatrix}}^{K_{(b)} = \mathrm{CX}\, K_{(a)}}$$

$$K_0 = K_1 = I/\sqrt{2} \qquad K_0 = I/\sqrt{2}, \quad K_1 = X/\sqrt{2}$$

Fig. 1. Constructing the pre-drop maps. The first two matrices apply H to t and allocate t in $|0\rangle$; their product is $K_{(a)}$. Applying `CX t v` gives $K_{(b)}$. In these two maps, columns are inputs $|0\rangle_v, |1\rangle_v$ and rows are ordered 00, 01, 10, 11, with t first. The upper row pair (blue) fixes $t = 0$, the lower pair (gold) fixes $t = 1$; these pairs are the slices. They are equal in program (a) but not proportional in program (b).

The specification holds exactly when every slice acts the same way up to a scalar, so $K_i = g_i W$. We therefore check proportionality of the slices rather than search for R and U. In program (a), both slices are $I/\sqrt{2}$ and $W = I$. In program (b), they are $I/\sqrt{2}$ and $X/\sqrt{2}$, which are not proportional.

When the block is hidden, these slices describe the remaining state by $\rho \mapsto \sum_i K_i \rho K_i^\dagger$, where ρ is the input density matrix. This is a *Kraus representation* of the output channel [15, §2.2.2]. Proportional slices mean that a single operator suffices to describe this channel, as $\rho \mapsto W \rho W^\dagger$. This is the standard Kraus-rank-one condition; the theorem below gives its program-level meaning through independent witnesses and local restoration.

In larger programs, W also produces other allocated registers, so it can have a larger output space than input space. It is an isometry, preserving norms and inner products. For contextual comparison we stop before the chosen drop and delete earlier drops, retaining every register in the joint map. We call this the *exposed prefix*. An earlier drop only removes a name, so its register may still be entangled with the chosen block. A context can then examine the block jointly with all the other registers, including those that an earlier drop would have hidden. Comparing only the final returned values would be weaker: an unused temporary and a temporary entangled with another register can both be dropped before returning `()`, leaving no output to distinguish them. The exposed comparison asks whether this block can be restored, without hiding that distinction.

Theorem 2.1 (the criterion, without measurement). *Work over $\mathbb{C}$, with every unitary available as a gate. Let K be the joint state map of the exposed prefix just before a block of temporaries is dropped in a well-typed measurement-free program, with the block first. Let ψ be the pure state in which the block was allocated. The following are equivalent.*

(1) The exposed prefix is contextually equivalent to a witness that prepares the block independently of the remaining computation.

(2) There are a fixed unit vector g on the block and a linear map W to all other registers, including those already dropped, such that $\forall x,\ Kx = g \otimes Wx$.

(3) One unitary on the block alone restores ψ at every input.

The map W is an isometry. The independent witness prepares g from ψ and realizes W by allocating the other temporaries and applying a unitary to the remaining registers.

We call g a *certificate* of safety, with coordinates g_i in $K_i = g_i W$; program (a) has $g = |+\rangle$ and $W = I$. The left of fig. 2 shows this separation. To return g to the allocation state ψ, consider their overlap $\langle \psi, g \rangle$.

- If the overlap is nonzero, choose the phase of g to make that overlap real and positive.
- If the overlap is zero, any phase works, so choose one that makes the first nonzero coordinate of g positive and real.

Call the chosen vector $\tilde{g}$. The inverse phase goes into W, so the joint map stays $K = \tilde{g} \otimes W$.

If $\tilde{g} = \psi$, no gate is needed. Otherwise, use as the return gate T the Householder reflection through the hyperplane orthogonal to $\psi - \tilde{g}$, which exchanges the two states [7, p. 340, Eqs. (3)–(4)] (right of fig. 2). Applied to the temporary alone, it returns it to ψ at every input and leaves the rest unchanged. Grover's diffusion step uses the same reflection form with the opposite overall sign [6, §4].

For program (a), this gives $T = H$, which also reverses the witness's choice $R = H$. Deutsch–Jozsa gives a less trivial example of the same separation. It uses one query to distinguish a black-box bit function promised to be constant or to return 0 on exactly half its inputs. Its oracle acts on both registers, but with the temporary prepared in $|-\rangle = (|0\rangle - |1\rangle)/\sqrt{2}$, its action is $O_f(|-\rangle \otimes |z\rangle) = (-1)^{f(z)}|-\rangle \otimes |z\rangle$. The temporary therefore stays in $|-\rangle$ for every data input, while the function's phase is carried by the data. Starting from an allocation in $|0\rangle$, it is safe though not back in $|0\rangle$, and its return gate leaves the data computation W intact. Its two slices differ by a minus sign, so requiring equal slices would reject this safe temporary. The phase kickback is part of the remaining computation W.

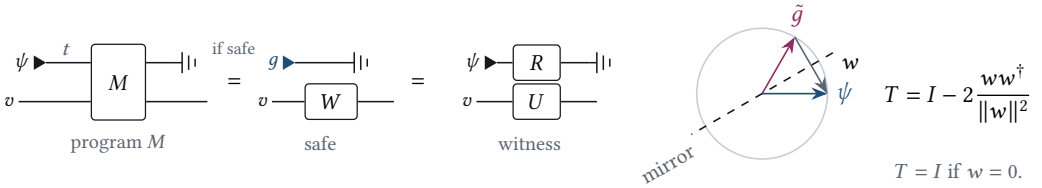


Fig. 2. For a safe drop, the joint maps before the ground symbols agree up to one overall phase. The left panel shows the program, its safe factorization and an independent witness that prepares g with R and implements W with U. Triangles prepare states and ground symbols mark drops. Fresh registers for U are omitted. Right, the return gate T exchanges $\tilde{g}$ and ψ. Its complex-state formula uses $w = \psi - \tilde{g}$ and conjugate transpose †.

3 One measurement record at a time

With measurement, the same program can be safe on one sequence of outcomes, or *record*, and unsafe on another, so we adapt the specification to each record. Our `lift` measures a register and binds the outcome as a classical parameter that only `if` guards can read. Making the result available to classical control follows the idea of *dynamic lifting* [4], but our language combines measurement and binding and has no separate circuit-generation

stage. Measurement keeps one group of coordinates and reports which group survived; its probability at a normalized input is the squared norm of the surviving part.

(c) *copy, measure, one joint gate*

```
prog c (v : [2]) : [2] {
  new t : [2] @ |0>;
  H v; CX v t;
  p = lift[{0}] t; V t v;
  drop t; return v }
```

In program (c), `lift[{0}]` tests membership in $\{0\}$, so it records 1 for $t = 0$ and 0 otherwise. Either outcome, when it occurs, leaves a product state at the lift. Choose V to keep $|00\rangle$ unchanged and send $|11\rangle$ to $(|01\rangle + |10\rangle)/\sqrt{2}$. The outputs are orthogonal unit vectors, so this action extends to a unitary. The same program is then safe on one record and unsafe on the other, as fig. 3 shows for input $x = \alpha|0\rangle + \beta|1\rangle$.

A record o lists the outcomes of the lifts executed before the drop. Write K_o for the unnormalized joint map at that site, retaining all registers as before. We again check whether the slices of K_o are proportional. For a block of temporaries, the drop is safe on o exactly when K_o separates into one fixed block state and its own remaining action W_o, with $K_o x = g \otimes W_o x$ for every input x. The state g may depend on the record, but must be fixed across its inputs. The unnormalized vector $W_o x$ keeps both the computation that survived and the weight it survived with. For a unit input x, the record's survival probability is $\|W_o x\|^2$. For example, on record 1, $W_{p=1} x = (\alpha + \beta)|0\rangle/\sqrt{2}$. Normalizing a nonzero $W_{p=1} x$ gives v's state $|0\rangle$, but hides how likely that record was. Input $|+\rangle$ reaches it with probability 1, whereas input $|0\rangle$ reaches it with probability $1/2$. The unnormalized map preserves these probabilities when comparing programs. A record that no input reaches has zero map, which is safe with $W_o = 0$ and any unit g. Each safe record therefore has its own return gate, selected by the outcomes already observed.

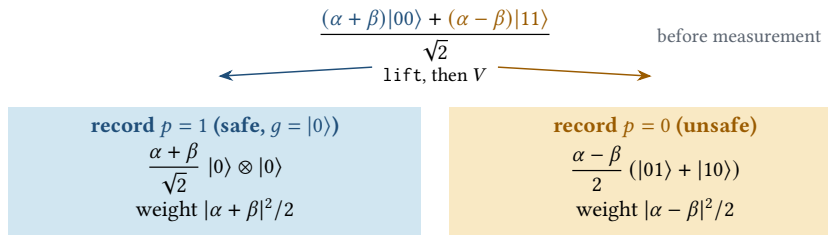


Fig. 3. In program (c), record $p = 1$ (blue) fixes the temporary in $|0\rangle$. Record $p = 0$ (gold) leaves an entangled pair when its weight is nonzero. Both records have weight $1/2$ on either basis input, but only $p = 1$ is safe.

A witness prepares $g = R\psi$ on the block independently of the input. On the remaining registers it allocates the other temporaries and a fresh qubit a in $|0\rangle$, applies a unitary U, then measures a and keeps outcome 1 of `lift[{0}]`. Omitting the fixed $|0\rangle_a$ factor leaves $g \otimes W_o$. This extra qubit and measurement realize W_o, a *contraction*, which cannot increase a vector's norm; without measurement, W is an isometry and needs neither. Figure 4 gives program (c)'s witness. We compare one selected program record o with the witness's kept record, using their joint states and original unnormalized weights. The bits are bound

inside each run and cannot be read by the context. The witness need not reproduce the program's other records, which this comparison does not examine.

```
prog witness (v : [2]) : [2] {
  new t : [2] @ |0>; new a : [2] @ |0>;
  H v; CX v a;
  p = lift[{0}] a; drop a;
  drop t; return v }
```

Keep record $p = 1$, where a is in $|0\rangle$. The exposed prefix retains a despite `drop a`. After omitting its fixed factor, the joint state immediately before `drop t` is $|0\rangle_t \otimes W_{p=1}x$. Here $R = I$, and U is `H v; CX v a`.

Fig. 4. A witness for program (c)'s safe record 1. The other outcome is auxiliary.

Theorem 3.1 (the criterion, one record at a time). *Work over $\mathbb{C}$, with every unitary available as a gate. Let s drop a block of temporaries, allocated in a pure state ψ, in a well-typed program. On a record o along which the program executes s, let K_o be the joint map of its exposed prefix, with the block first. The following are equivalent ways to say that the block is safe on o. Clause (4) is the observational specification, clause (2) its factorization test, and clause (3) its witness.*

(1) The block is safe on o.

(2) There are a unit vector g on the block and a linear contraction W_o to all other registers such that: $\forall x,\quad K_o x = g \otimes W_o x,\quad \|W_o x\| \le \|x\|.$

(3) A one-lift witness has map K_o on its kept record, after omitting its fixed $|0\rangle_a$ factor.

(4) For some such witness, no closed context distinguishes the exposed prefix on o from the witness's kept record, with that fixed factor omitted. Measurement-free contexts suffice, with their outputs observed at the end.

When $K_o = 0$, all four hold: no input reaches the drop on that record.

Since reuse must preserve observable behavior, we might hope to decide safety from the program's surviving outputs and the probabilities of its measurement records. But the program's summed output channel and record weights do not determine safety. Summing merges the records' separate actions on the outputs; it does not determine the individual maps, even when their weights are known. Two programs can agree on the sum and on every record's weight yet have opposite verdicts on every record. Our test keeps the joint map on each record, including the block and all other registers.

4 Blocks and loans

At a tuple drop, the specification asks for independent preparation of the whole block, so we test all the named registers together. They can be entangled with each other: a fixed Bell pair $(|00\rangle + |11\rangle)/\sqrt{2}$ is safe as a block, but each half has the mixed state $I/2$ and remains entangled with the other, so neither is safe separately. Testing components separately at the same point asks for a product certificate, and a fixed mixed state does not suffice.

We also consider borrowed or dirty registers, which we call *loans*. A loan can arrive in an unknown state entangled with the caller. Without measurement, returning it must preserve that state and entanglement. On a measurement record, the *restored* map acts as the identity on the loan input, tensored with the unnormalized action on the other

inputs. If the caller supplied a loan entangled with another input, a lift of that input can still condition the loan, even when the restored map has this identity factor. In `mixed` below, `drop (l,t)` groups a loan and a temporary. The loan is handed back after the inserted restoration; `()` means that neither register is a program output. Such a block need not end in one fixed state. Following the same intuition as before, returning a loan is safe when its action on the loan input separates from the computation on the other inputs. We use this factorization as the safe-return contract for loans. At a safe drop without measurement, the map factors as $B \otimes W$, where B is an isometric encoding of the loan inputs in the block and W acts on the other inputs. With no loans, B reduces to the fixed state g. The desired placement J puts the unchanged loans beside the temporaries in their allocation states, and a return gate T undoes the encoding by satisfying $TB = J$. In `mixed`, $B\ell = X\ell \otimes |+\rangle$ and $J\ell = \ell \otimes |0\rangle$, so $T = X \otimes H$ returns the loan and restores the temporary, as the diagram shows.

```
prog mixed (loan l : [2]) : 1 {
  new t : [2] @ |0>;
  H t; X l;
  drop (l,t); return () }
```

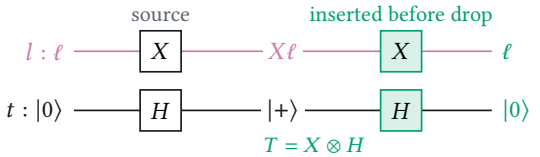


Fig. 5. The inserted $T = X \otimes H$, with the loan first, returns every loan input ℓ and restores t to $|0\rangle$. The source `drop` applies no gate.

Since B preserves norms and inner products, one unitary on the block can undo the encoding and restore the temporaries beside the unchanged loans (fig. 5). For a loan-only block, B is unitary and the return gate is its inverse $B^\dagger$.

5 Scope, implementation and open questions

We implemented a typed first-order language with finite-dimensional registers and classical conditionals. We prove type safety for its core calculus and abstract machine. For a pooled machine that reuses safely reclaimed whole registers in their allocation states, we also show that reuse preserves records, their weights and the reduced output state. This theorem concerns that machine; the allocator described below pools individual qubits and is tested by simulation rather than proved here. For qubit programs over X, `CX` and Toffoli with basis-state allocations, deciding safety of temporary-only drops is coNP-complete in program-text size. This holds for a chosen site on a record along which it is executed, and for all such sites on all records. Hardness already holds for a single drop without measurement. Thus, unless P = NP, no polynomial-time check on this fragment is both sound and complete.

Our Haskell implementation has a matrix checker for a one-temporary catalogue and an independent state-vector checker for blocks, records and loans; both grow exponentially with the qubit count. The input syntax restricts amplitudes to the field generated over $\mathbb{Q}$ by a primitive sixteenth root of unity. The OpenQASM 3 emitter handles power-of-two dimensions and emits basis-state preparations. It supports a table of named gates and diagonal, basis-permutation and one-qubit matrix literals, with conditionals whose branches vary only gates and the values they return. Program (c)'s joint gate V is outside these forms, so the emitter refuses that program. Non-basis preparations are assumed rather

than emitted and must be supplied at circuit input. The compiler computes exact safety verdicts and synthesizes return gates before execution, selecting a gate by the outcomes already observed. It emits restoration for safe temporary blocks unless a numerical check refuses the whole program with an error naming the drop. For loans, it recognizes factors proportional to the identity, a one-qubit gate or a basis permutation, including mixed blocks when such a factor separates from a fixed temporary state. The theory guarantees a return unitary for each safe record at a site, but broader practical reuse requires further work.

- For programs it accepts, the allocator restores and reuses basis-allocated temporary blocks safe on all reachable records, clearing them to zero before later basis-state allocations. It uses one wire map across records; reuse available on only some records remains open.
- A loan drop after a measurement or inside or after a conditional makes the compiler refuse the program. For earlier loan drops, safe factors outside the supported shapes receive no return gate, and the circuit marks the omission in a comment. Unsafe drops likewise emit no restoration and leave the wire live. Successful emission therefore does not guarantee that every loan is restored. Synthesizing return circuits for general loan factors remains open; the ideal return gate preserves caller entanglement at the drop, subject to the measurement conditioning described above.
- Emitted gate parameters are numerical, whereas the theorems assume ideal unitaries. Simulation checks cover tested records; a proved error bound would give a guarantee beyond them.

The circuit simulations compare emitted outputs with a reference that shares the factor-extraction code, so they check emission and reuse rather than independently validating that extraction.

Existing uncomputation checks can reject programs even when restoration is possible. For temporaries, our criterion is necessary and sufficient for restoration by a unitary on the dropped block alone, within the stated ideal-gate model. It checks the resulting computation rather than how the program was written, though our implementations of this test are exponential. Joint uncomputation can also restore an entangled temporary by acting on other registers. The particular checks below ask different questions and can accept different programs; this does not make block-local restoration more general than joint uncomputation.

- Silq and Unqomp provide sufficient conditions for their uncomputation transformations, while ReQWIRE verifies assertions that ancillas have been restored [1, 12, 13]. Unqomp, for example, rejects two successive Hadamards on an ancilla because Hadamard can create superposition from a basis state, even though together they leave the ancilla unchanged [12, §6.2]. Our test accepts this case because it checks the resulting map. Qutes tracks entanglement and liveness and uses adjoint histories for reclamation [3, Defs. 2–6]. Li et al.'s ancilla-safety verifier also repairs separable errors on individual dirty ancillas with local rotations [9, §5.2, Alg. 2]. Here we ask whether an entire dropped block is restorable, record by record.

- Q# release and fractional types require restoration, with fractional types treating reversible classical ancilla bits [2, 14]. Quipper's assertive termination requires a specified basis state and leaves the assertion to the programmer [5, §4.2.2]. Q# does not enforce its release condition at compile time, although a simulator can check it at runtime [11].
- Twist combines static analysis with runtime purity checks [17]. A temporary may be pure at each unentangled input while its state still depends on that input, as after a swap; our test requires one fixed state. Guppy tracks ownership and borrowing [8], while Bona schedules borrows whose circuit already satisfies its safe-return condition [16].
- Liu, Zhou and Meng's functionality test [10, Def. 3.1] treats the sum of the slices as the remaining operation and asks whether it is unitary. Our criterion instead asks whether all slices are proportional to the same operation. Program (a) is safe, but its slice sum $\sum_i K_i = \sqrt{2}I$ is only proportional to a unitary, since it multiplies norms by $\sqrt{2}$. Conversely, `new t @ |0>; CX v t; drop t` has slices $|0\rangle\langle 0|$ and $|1\rangle\langle 1|$. Their sum is I, but they are not proportional. Another `CX v t` restores the temporary by joint uncomputation, whereas no gate on t alone can do so for every input. Their coNP-completeness result concerns the reversible-circuit uncomputation problem [10, Cor. 3.1]; ours concerns restoration on the dropped block alone.

The present language has no loops or recursion. How should a type system certify safe re-entry into temporary scopes in loops and quantum recursion, including quantum guards' scopes? When is unitary restoration preferable to measurement and reset once gate cost, latency and noise are accounted for?

References

[1] Benjamin Bichsel, Maximilian Baader, Timon Gehr, and Martin Vechev. 2020. Silq: A High-Level Quantum Language with Safe Uncomputation and Intuitive Semantics. In *Proceedings of the 41st ACM SIGPLAN Conference on Programming Language Design and Implementation*. 286–300. doi:10.1145/3385412.3386007

[2] Chao-Hong Chen, Vikraman Choudhury, Jacques Carette, and Amr Sabry. 2020. Fractional Types: Expressive and Safe Space Management for Ancilla Bits. In *Reversible Computation — 12th International Conference, RC 2020 (Lecture Notes in Computer Science, Vol. 12227)*. Springer, 169–186. doi:10.1007/978-3-030-52482-1_10

[3] Simone Faro, Francesco Pio Marino, and Gabriele Messina. 2026. Reversible Lifetime Semantics for Quantum Programs. arXiv:2603.14538 [quant-ph]

[4] Peng Fu, Kohei Kishida, Neil J. Ross, and Peter Selinger. 2023. Proto-Quipper with Dynamic Lifting. *Proceedings of the ACM on Programming Languages* 7, POPL (2023). doi:10.1145/3571204

[5] Alexander S. Green, Peter LeFanu Lumsdaine, Neil J. Ross, Peter Selinger, and Benoît Valiron. 2013. Quipper: A Scalable Quantum Programming Language. *ACM SIGPLAN Notices (PLDI)* 48, 6 (2013), 333–342. doi:10.1145/2499370.2462177

[6] Lov K. Grover. 1996. A Fast Quantum Mechanical Algorithm for Database Search. In *Proceedings of the Twenty-Eighth Annual ACM Symposium on the Theory of Computing (STOC)*. ACM, 212–219. doi:10.1145/237814.237866

[7] Alston S. Householder. 1958. Unitary Triangularization of a Nonsymmetric Matrix. *J. ACM* 5, 4 (Oct. 1958), 339–342. doi:10.1145/320941.320947

[8] Mark Koch, Agustín Borgna, Craig Roy, Alan Lawrence, Kartik Singhal, Seyon Sivarajah, and Ross Duncan. 2025. Imperative Quantum Programming with Ownership and Borrowing in Guppy. arXiv:2510.13082 [cs.PL] Presented at PLanQC 2025.

[9] Jiqi Li, Jingyi Mei, Wang Fang, and Ji Guan. 2026. Formal Verification of Quantum Ancilla Safety. In *Computer Aided Verification*. Springer Nature Switzerland, 326–348. doi:10.1007/978-3-032-32537-2_16 arXiv:2608.13099.

[10] Chenke Liu, Li Zhou, and Boning Meng. 2026. Quantum Uncomputation of Clean and Dirty Ancilla Qubits. arXiv:2608.09578 [cs.PL]

[11] Microsoft. 2025. Quantum Memory Management in Q#. Microsoft Learn, Azure Quantum documentation. https://learn.microsoft.com/en-us/azure/quantum/user-guide/language/statements/quantummemorymanagement

[12] Anouk Paradis, Benjamin Bichsel, Samuel Steffen, and Martin Vechev. 2021. Unqomp: Synthesizing Uncomputation in Quantum Circuits. In *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI '21)*. ACM, 222–236. doi:10.1145/3453483.3454040

[13] Robert Rand, Jennifer Paykin, Dong-Ho Lee, and Steve Zdancewic. 2019. ReQWIRE: Reasoning about Reversible Quantum Circuits. *Electronic Proceedings in Theoretical Computer Science* 287 (2019), 299–312. doi:10.4204/EPTCS.287.17

[14] Krysta Svore, Alan Geller, Matthias Troyer, John Azariah, Christopher Granade, Bettina Heim, Vadym Kliuchnikov, Mariia Mykhailova, Andres Paz, and Martin Roetteler. 2018. Q#: Enabling Scalable Quantum Computing and Development with a High-level DSL. In *Proceedings of the Real World Domain Specific Languages Workshop 2018 (RWDSL2018)*. Article 7, 10 pages. doi:10.1145/3183895.3183901

[15] John Watrous. 2018. *The Theory of Quantum Information*. Cambridge University Press. doi:10.1017/9781316848142 Theorem 2.22 and Corollaries 2.23, 2.27 for the operator-sum form and its freedom; the Choi rank is defined on p. 78 and isometric channels on p. 91.

[16] Xiaoquan Xu, Chenke Liu, Boning Meng, Zihao Shen, and Li Zhou. 2026. Bona: Automatic Management of Dirty Ancilla Borrowing in Quantum Circuits. arXiv:2608.08765 [cs.PL]

[17] Charles Yuan, Christopher McNally, and Michael Carbin. 2022. Twist: Sound Reasoning for Purity and Entanglement in Quantum Programs. *Proceedings of the ACM on Programming Languages* 6, POPL, Article 30 (2022), 32 pages. doi:10.1145/3498691